

**B.TECH FINAL YEAR CAPSTONE PROJECT**

# **Building a Retrieval-Augmented Generation (RAG) Application with Open-Source LLMs**

*A short, practical build guide — no prior coding experience needed*

Department of Computer Science & Engineering

Duration: 4 Weeks | Team Project

# 1. Problem Statement & Project Overview

---

## 1.1 The Problem

Large Language Models (LLMs) can write fluent, confident answers — but they only know what they saw during training, cannot reliably cite where an answer came from, and often “hallucinate” facts that sound correct but aren't. This makes a plain LLM unreliable as a knowledge assistant for anything specific: a college's own rules, a company's own manuals, or notes from a particular course.

## 1.2 The Solution: Retrieval-Augmented Generation (RAG)

RAG solves this by connecting an LLM to a specific set of documents. When a user asks a question, the system first retrieves the most relevant pieces of those documents, then gives them to the LLM as context before it answers. The answer is now grounded in real source material instead of the model's memory — and the system can show exactly which document it used.

## 1.3 What You Will Build

A team-based Knowledge Assistant: a chatbot that answers questions accurately from a set of documents your team chooses — for example, your college handbook, a subject's lecture notes, a company's product manual, or research papers on a topic you're interested in. The finished system must:

- Take in a set of source documents (PDFs or text files) as its knowledge base.
- Split and convert that content into a searchable vector index.
- Retrieve the most relevant content for any question a user asks.
- Use an open-source LLM to generate an answer grounded in that retrieved content.
- Show the source of every answer, and honestly say “I don't know” when the answer isn't in the documents.
- Run through a simple, working chat interface in the browser.

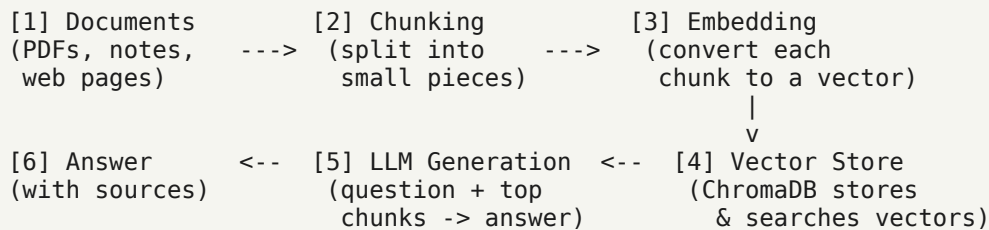
**Why this project matters:** RAG is the most common real-world way LLMs are actually deployed in industry — in support bots, internal search tools, and research assistants. Building one yourself is the fastest way to turn what you already know about RAG concepts into a system you can demo and explain.

## 1.4 Team & Duration

Work in a team of your choice, over 4 weeks. How you divide the work among your teammates (data collection, coding, testing, documentation) is entirely up to your team — the only requirement is that every member understands and can explain the full pipeline by the end.

## 2. How RAG Works: Concepts & Architecture

Every RAG system, no matter how advanced, is built from the same six stages. Understanding this diagram is the key to understanding — and building — your whole project.



### 1. Document Ingestion

Collect the raw material your assistant should know about — PDFs, text files, or web pages. Start with 10–30 documents; relevance matters more than volume.

### 2. Chunking

Models can only read a limited amount of text at once, so documents are split into smaller, overlapping pieces (roughly 300–800 words each) so no idea gets cut off mid-sentence.

### 3. Embedding

Each chunk is converted into a numeric vector that captures its meaning. Chunks about similar topics end up mathematically close together.

### 4. Vector Storage

These vectors are stored in a vector database (ChromaDB) that can instantly find the chunks most similar to any new question.

### 5. Retrieval + Generation

When a user asks a question, it is embedded the same way, the most relevant chunks are retrieved, and both the question and those chunks are given to an open-source LLM to generate an answer.

### 6. Answer with Sources

The interface shows the answer plus which document it came from, so it can be verified.

**Key idea to remember:** A RAG system is only as good as its retrieval. A great LLM with poor retrieval still gives poor answers — so most of your debugging time will go into Stages 1–4, not the LLM itself.

### 2.1 Tools You Will Use (all free)

| Purpose            | Tool                          | Notes                                                    |
|--------------------|-------------------------------|----------------------------------------------------------|
| Coding environment | Google Colab                  | Free, browser-based, nothing to install                  |
| Open-source LLM    | Llama 3.1 / Mistral / Gemma 2 | Accessed via Groq's free API, or run locally with Ollama |
| Embedding model    | all-MiniLM-L6-v2              | Small and fast, runs on any laptop CPU                   |

|                 |            |                                                     |
|-----------------|------------|-----------------------------------------------------|
| Vector database | ChromaDB   | No server setup needed                              |
| RAG framework   | LlamaIndex | Handles most of the pipeline in a few lines of code |
| Web interface   | Streamlit  | Turns a Python script into a chat app               |

### 3. Step-by-Step Build Guide

Every code block below is complete and runnable in Google Colab. Read the explanation before each block, run it, look at the output, then move to the next. Do not skip ahead — each stage depends on the previous one's output.

#### 3.1 Setup

Create one Google Colab notebook for your project. In the first cell, install the libraries you need:

```
!pip install -q llama-index llama-index-embeddings-huggingface \
  llama-index-llms-groq chromadb llama-index-vector-stores-chroma \
  streamlit pypdf
```

#### 3.2 Loading Your Documents

Upload your PDFs/text files into the data/ folder (in Colab, use the folder icon on the left sidebar to upload, or mount Google Drive). Then load them with LlamaIndex's built-in reader — it handles PDFs, .txt, and .docx automatically:

```
from llama_index.core import SimpleDirectoryReader

# Loads every supported file inside the data/ folder
documents = SimpleDirectoryReader("data").load_data()

print(f"Loaded {len(documents)} document(s)")
print(documents[0].text[:500]) # preview the first 500 characters
```

What this does: SimpleDirectoryReader opens every file in the folder and turns each one into a “Document” object holding its raw text plus metadata (like the filename), which we will need later to show sources.

#### 3.3 Chunking the Documents

LlamaIndex calls chunks “nodes.” A SentenceSplitter breaks text on sentence boundaries so chunks stay coherent, with overlap so context is not lost between chunks.

```
from llama_index.core.node_parser import SentenceSplitter

splitter = SentenceSplitter(
    chunk_size=512,      # max tokens per chunk
    chunk_overlap=64,    # tokens shared between consecutive chunks
)

nodes = splitter.get_nodes_from_documents(documents)
print(f"Created {len(nodes)} chunks from {len(documents)} documents")
print(nodes[0].text)    # inspect the first chunk
```

**Team task:** As a group, read 5 random chunks. If a chunk looks cut off mid-idea or mixes two unrelated topics, adjust chunk\_size or chunk\_overlap and re-run. Record your final choice and reasoning in the report.

### 3.4 Generating Embeddings

We convert each chunk into a vector using a small, free, CPU-friendly embedding model from Hugging Face.

```
from llama_index.embeddings.huggingface import HuggingFaceEmbedding

embed_model = HuggingFaceEmbedding(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

# quick sanity check
test_vector = embed_model.get_text_embedding("What is retrieval augmented generation?")
print(f"Embedding length: {len(test_vector)}") # should print 384
```

### 3.5 Building the Vector Index with ChromaDB

Now we store every chunk's embedding in ChromaDB and wrap it as a LlamaIndex "VectorStoreIndex," which handles both storage and similarity search for us.

```
import chromadb
from llama_index.vector_stores.chroma import ChromaVectorStore
from llama_index.core import VectorStoreIndex, StorageContext, Settings

Settings.embed_model = embed_model # tell LlamaIndex which embedder to use everywhere

chroma_client = chromadb.PersistentClient(path="./chroma_db")
chroma_collection = chroma_client.get_or_create_collection("capstone_knowledge_base")
vector_store = ChromaVectorStore(chroma_collection=chroma_collection)
storage_context = StorageContext.from_defaults(vector_store=vector_store)

index = VectorStoreIndex(nodes, storage_context=storage_context)
print("Vector index built and saved to ./chroma_db")
```

Test retrieval on its own, before adding the LLM, so you can judge retrieval quality in isolation:

```
retriever = index.as_retriever(similarity_top_k=3)
results = retriever.retrieve("What is the attendance policy?")

for r in results:
    print(f"Score: {r.score:.3f} | Source: {r.node.metadata.get('file_name')}")
    print(r.node.text[:200], "\n---")
```

### 3.6 Connecting an Open-Source LLM

Two supported options — pick whichever fits your team's hardware:

#### Option A (recommended): Groq API — free, fast, no local GPU needed

- Create a free account and API key at [console.groq.com](https://console.groq.com).
- In Colab, store the key safely using the Secrets panel (key icon in the left sidebar) rather than pasting it into code.

```
from google.colab import userdata
from llama_index.llms.groq import Groq

llm = Groq(
    model="llama-3.1-8b-instant",
    api_key=userdata.get("GROQ_API_KEY")
)
Settings.llm = llm

response = llm.complete("In one sentence, what is RAG?")
print(response)
```

### Option B: Ollama — fully local, works offline

- Install Ollama on a laptop with at least 16GB RAM: [ollama.com/download](https://ollama.com/download).
- Pull a small open model: `ollama pull llama3.1:8b` (or `ollama pull mistral`).

```
from llama_index.llms.ollama import Ollama

llm = Ollama(model="llama3.1:8b", request_timeout=120.0)
Settings.llm = llm
```

## 3.7 The Complete RAG Query Engine

With `embed_model`, `index`, and `llm` all set, `LlamaIndex` can assemble the full retrieve-then-generate pipeline in one line:

```
query_engine = index.as_query_engine(similarity_top_k=3)

response = query_engine.query("What is the attendance policy?")
print("ANSWER:\n", response)

print("\nSOURCES USED:")
for node in response.source_nodes:
    print("-", node.metadata.get("file_name"), f"(score {node.score:.3f})")
```

Custom prompt template (recommended): controlling the exact instructions given to the LLM makes answers more reliable and lets you enforce “only answer from context” behavior:

```
from llama_index.core import PromptTemplate

qa_template = PromptTemplate(
    "You are a helpful assistant that answers ONLY using the context below.\n"
    "If the answer is not contained in the context, say "\n"
    "'I don't have enough information to answer that.'\n\n"
    "Context:\n{context_str}\n\n"
    "Question: {query_str}\n"
    "Answer:"
)

query_engine = index.as_query_engine(
    similarity_top_k=3,
    text_qa_template=qa_template
)
```

**No-code fallback (Flowise):** If your team is completely stuck on Python, Flowise ([flowiseai.com](https://flowiseai.com), open source, self-hostable) provides a drag-and-drop canvas with pre-built nodes for document loaders, ChromaDB, embeddings, and

open-source LLMs, connected by drawing wires instead of writing code. The underlying RAG concepts you explain in your report are identical either way — but the coded path in 3.1–3.6 is what we recommend and will assess more favorably, since it demonstrates deeper engineering understanding.

### 3.8 Building the Web Interface (Streamlit)

Save the following as `app.py`. It reuses everything built above inside a simple chat UI.

```
import streamlit as st
from llama_index.core import VectorStoreIndex, Settings, StorageContext, PromptTemplate
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from llama_index.llms.groq import Groq
from llama_index.vector_stores.chroma import ChromaVectorStore
import chromadb, os

st.set_page_config(page_title="Team RAG Knowledge Assistant", page_icon="📖")
st.title("📖 Team RAG Knowledge Assistant")

@st.cache_resource
def load_query_engine():
    Settings.embed_model = HuggingFaceEmbedding(
        model_name="sentence-transformers/all-MiniLM-L6-v2")
    Settings.llm = Groq(model="llama-3.1-8b-instant",
        api_key=os.environ["GROQ_API_KEY"])
    client = chromadb.PersistentClient(path="./chroma_db")
    collection = client.get_or_create_collection("capstone_knowledge_base")
    vector_store = ChromaVectorStore(chroma_collection=collection)
    index = VectorStoreIndex.from_vector_store(vector_store)
    return index.as_query_engine(similarity_top_k=3)

query_engine = load_query_engine()

if "history" not in st.session_state:
    st.session_state.history = []

question = st.chat_input("Ask a question about the knowledge base...")
if question:
    response = query_engine.query(question)
    sources = sorted({n.metadata.get("file_name", "unknown") for n in response.source_nodes})
    st.session_state.history.append((question, str(response), sources))

for q, a, srcls in st.session_state.history:
    with st.chat_message("user"):
        st.write(q)
    with st.chat_message("assistant"):
        st.write(a)
    st.caption("Sources: " + ", ".join(srcls))
```

Run it locally (or on a classmate's machine) with:

```
export GROQ_API_KEY="your-key-here"
streamlit run app.py
```

## 4. Evaluation, Deliverables & Resources

### 4.1 How You Will Be Evaluated

| Criterion                     | Marks | What is Assessed                                                                       |
|-------------------------------|-------|----------------------------------------------------------------------------------------|
| RAG Pipeline Correctness      | 25    | Ingestion, chunking, embedding, indexing, and retrieval all work end-to-end            |
| Answer Quality & Groundedness | 25    | Answers are accurate, cite correct sources, and say “I don't know” instead of guessing |
| Working Demo                  | 20    | Chat interface runs live and answers real-time questions correctly                     |
| Code Quality                  | 15    | Code runs without errors and is reasonably organized                                   |
| Documentation                 | 15    | Report clearly explains what was built, key decisions, and results                     |

Before submitting, test your system on at least 15 questions: some with direct answers in the documents, some that need combining information from multiple chunks, and a few that are deliberately outside the knowledge base (to confirm the system says it doesn't know, instead of making something up).

### 4.2 What to Submit

- Source code (Colab notebook or repository link).
- The working Streamlit chat application.
- A short report (4–6 pages): what you built, the architecture, your 15-question test results, and challenges you faced.
- A short live demo (or a 3–5 minute backup video).

### 4.3 Suggested Pace (4 Weeks)

| Week | Goal                  | Focus                                                                       |
|------|-----------------------|-----------------------------------------------------------------------------|
| 1    | Data ready            | Pick your topic, collect documents, get chunking working (Sections 3.1–3.3) |
| 2    | Retrieval working     | Embeddings and ChromaDB index built and tested (Sections 3.4–3.5)           |
| 3    | Full pipeline working | LLM connected, end-to-end answers with sources (Sections 3.6–3.7)           |
| 4    | Polish & present      | Chat UI, testing, report, and demo rehearsal (Section 3.8)                  |

### 4.4 Key Resources

- [LlamaIndex documentation — https://docs.llamaindex.ai/](https://docs.llamaindex.ai/)

- [Groq — free API for open-source LLMs — https://console.groq.com/](https://console.groq.com/)
- [Ollama — run open-source LLMs locally — https://ollama.com/](https://ollama.com/)
- [ChromaDB documentation — https://docs.trychroma.com/](https://docs.trychroma.com/)
- [Streamlit documentation — https://docs.streamlit.io/](https://docs.streamlit.io/)
- [Original RAG paper \(Lewis et al., 2020\) — https://arxiv.org/abs/2005.11401](https://arxiv.org/abs/2005.11401)

## 4.5 Common Issues

- Irrelevant retrieved chunks → adjust chunk size (try 256–768) or check your source documents actually cover the test question.
- LLM answers using outside knowledge instead of your documents → tighten the prompt template in Section 3.7 to force “only use the given context.”
- Groq API rate-limit errors → add a short pause between test queries.
- Stale or duplicate results after changing chunking → delete the local chroma\_db folder and rebuild the index.